

TENURE

Complete System Architecture Blueprint

Simon Business School Fall 2026 Pilot

Design objective

Build a secure, multi-tenant operating system where university oversight, club governance, role-based memory, approvals, events, and communication operate in one coherent infrastructure.

30+ clubs

3-5 board members / club

OSE full oversight

Role-based access

G GOVERNANCE

OSE Administration
Presidents
Functional VPs
Members

P CORE PLATFORM

Approvals
Unified calendar
Messaging / InMail
Documents and memory

I INFRASTRUCTURE

Multi-tenant data
Policy enforcement
Audit and observability
Cloud deployment

Prepared as the canonical product, data, security, and deployment blueprint for Tenure.

Source context: Simon OSE discovery discussion and questionnaire, July 2026.

Executive architecture decisions

Tenure is an institutional operating system, not merely a club directory. The architecture must preserve continuity across annual leadership turnover while giving OSE complete oversight and keeping each club securely isolated.

1

1. Start as a modular monolith

One deployable backend with strict domain modules. This reduces pilot risk while preserving clean extraction boundaries for future services.

2

2. Treat role seats as durable identities

President, VP Finance, VP Marketing, and other seats persist. People occupy them for a term; the memory and responsibilities remain.

3

3. Enforce RBAC plus context rules

Role permissions are combined with tenant, club, workflow, record sensitivity, and relationship checks. OSE has full institutional access.

4

4. Make approvals first-class workflow objects

VP requests route to the President; President escalations route to OSE. Every state change is timestamped, messaged, and audited.

5

5. Build one calendar truth

All approved and proposed events appear in a shared calendar with conflict detection, role-aware visibility, and approval status.

6

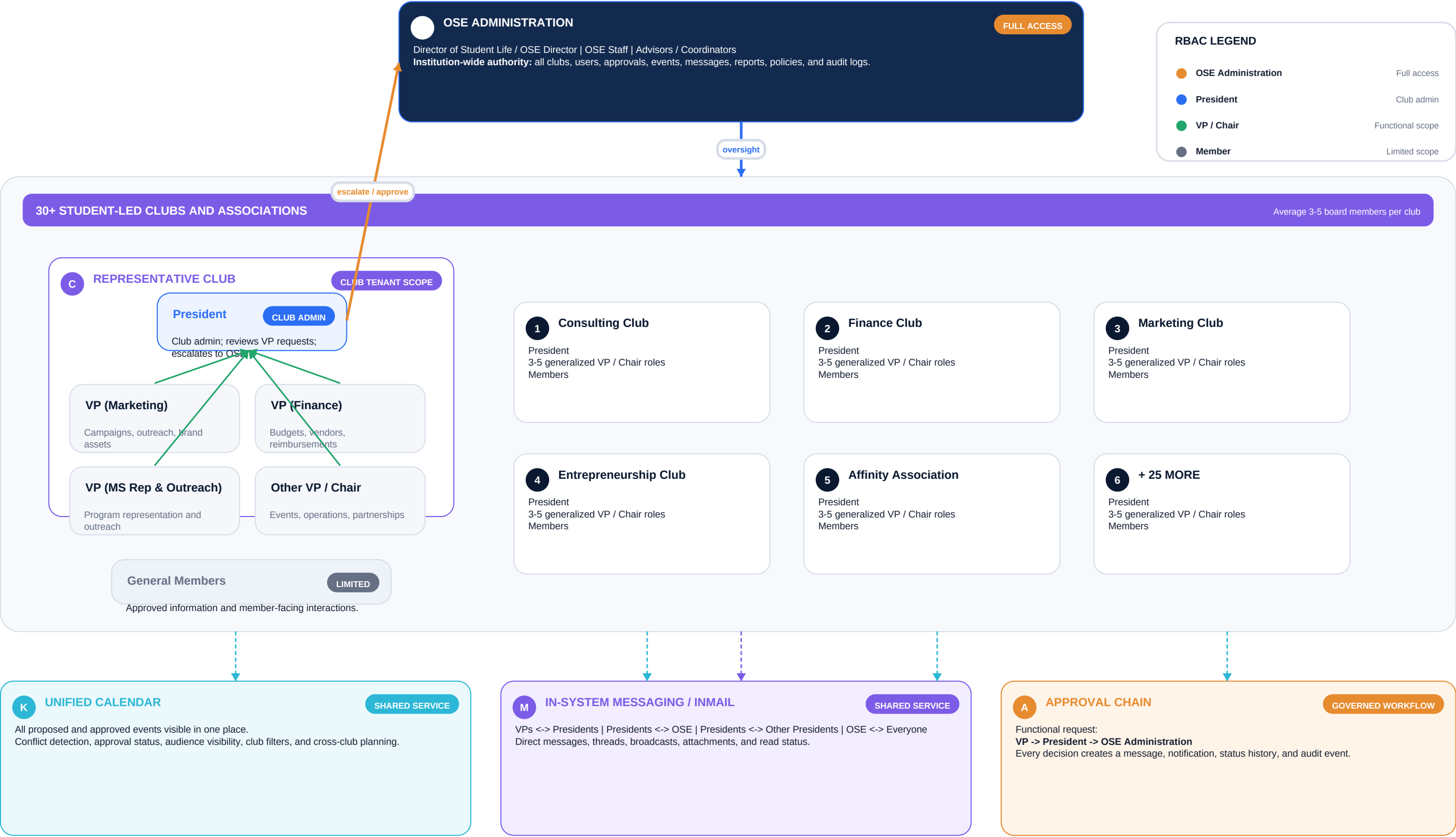
6. Keep communication inside the record

Messages, approval threads, announcements, and cross-club coordination stay linked to clubs, events, requests, and users.

Pilot-scale constraints and architectural response

Constraint	Architecture response
30+ clubs; roughly 90-150 board officers	Single school tenant, club-scoped data, indexed queries, and role templates. No distributed microservices are needed for this scale.
Annual turnover and inconsistent handoffs	Role-seat history, immutable assignments, structured handoff records, and source-cited institutional memory.
OSE needs complete visibility	Institution-level administrator role, full audit access, override paths, and global calendar/message privileges.
University integrations may be slow	Core product works independently; SSO, Outlook/Google, Box, and SimonSource are adapters added after the pilot.

Organizational hierarchy and RBAC - one view

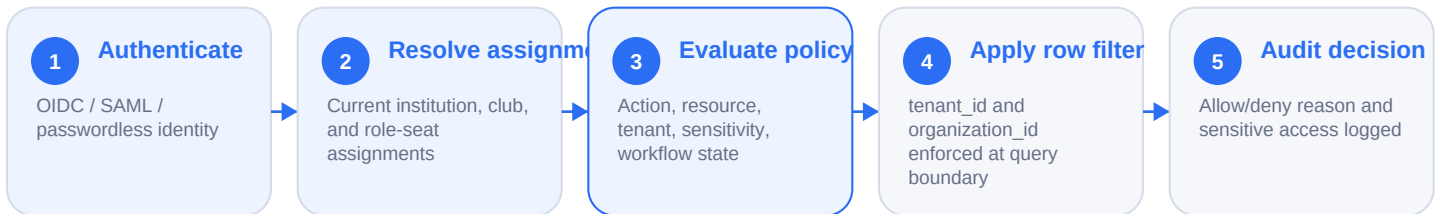


Roles, access scopes, and policy model

Tenure should resolve authorization as: **user role + tenant + organization + resource sensitivity + workflow state + relationship to the record**. This prevents broad role labels from becoming accidental data leaks.

Role	Default scope	Key capabilities	Restrictions
OSE Director / Admin	Institution / school	All clubs and users; approve or override; global calendar; broadcasts; policy and audit access	No hidden club records; all sensitive actions audited
OSE Staff / Advisor	All clubs or assigned clubs	Review requests; comment; access club history; create OSE tasks and announcements	Cannot change tenant security policy unless elevated
Club President	One club	Manage board roster; approve VP requests; submit to OSE; see club calendar, messages, files, budgets	Cannot access another club private data or OSE-only notes
VP / Chair	One club + functional domain	Create requests; manage domain records; message President; view club context	Cannot approve own request or access unrelated sensitive domains
General Member	Member-facing club scope	View approved events and content; receive announcements; participate in permitted threads	No private financial, approval, handoff, or administrative records

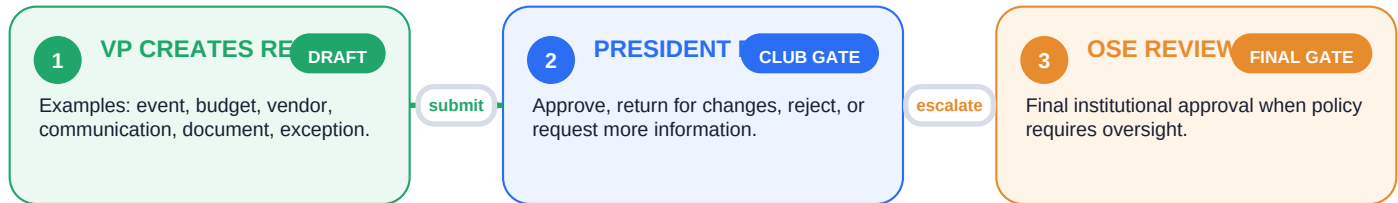
Authorization decision pipeline



Sensitive object rules

Object	VP	President	OSE	Rule
Budget / reimbursement	Finance VP only	Read / approve club stage	Full	Field-level masking may apply
Advisor notes	No	No unless shared	Full	OSE-only by default
Approval request	Creator / assigned VPs	Club-wide review	Full	Immutable state history
Tenure - System Architecture Blueprint Club messages	Thread participants	Club threads	Full with audit	No silent access; read events logged

Approval workflow architecture



Canonical state machine



Atomic side effects on every transition

1 Status history

old_state, new_state, actor, timestamp, reason

2 Message thread

System entry added to the request conversation

3 Notifications

In-app plus email digest / urgent mail rules

4 Calendar update

Event proposal or approval reflected immediately

5 Audit event

Tamper-evident log for compliance and debugging

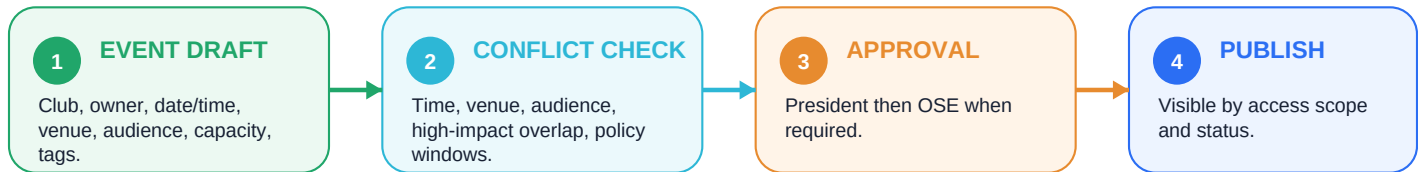
6 Search index

Updated permission-aware search document

Workflow invariants

- **No self-approval:** the creator cannot satisfy the next required gate.
- **No skipped gate:** OSE approval cannot occur before the President stage unless an authorized override records a reason.
- **Immutable history:** comments may be corrected through a new entry, not by silently rewriting the audit trail.
- **Permission snapshot:** decision records store the role and policy context at the time of action.

Unified calendar system



Calendar views and audiences

1 OSE global view

All clubs, proposed + approved events, conflicts, status, owners, venues.

2 Club board view

Own club events plus relevant institutional schedule and conflict warnings.

3 Member view

Approved events visible to the member or selected audience.

4 Cross-club planning

Presidents can inspect shared availability and coordinate before submission.

Conflict engine - hard vs. soft conflicts

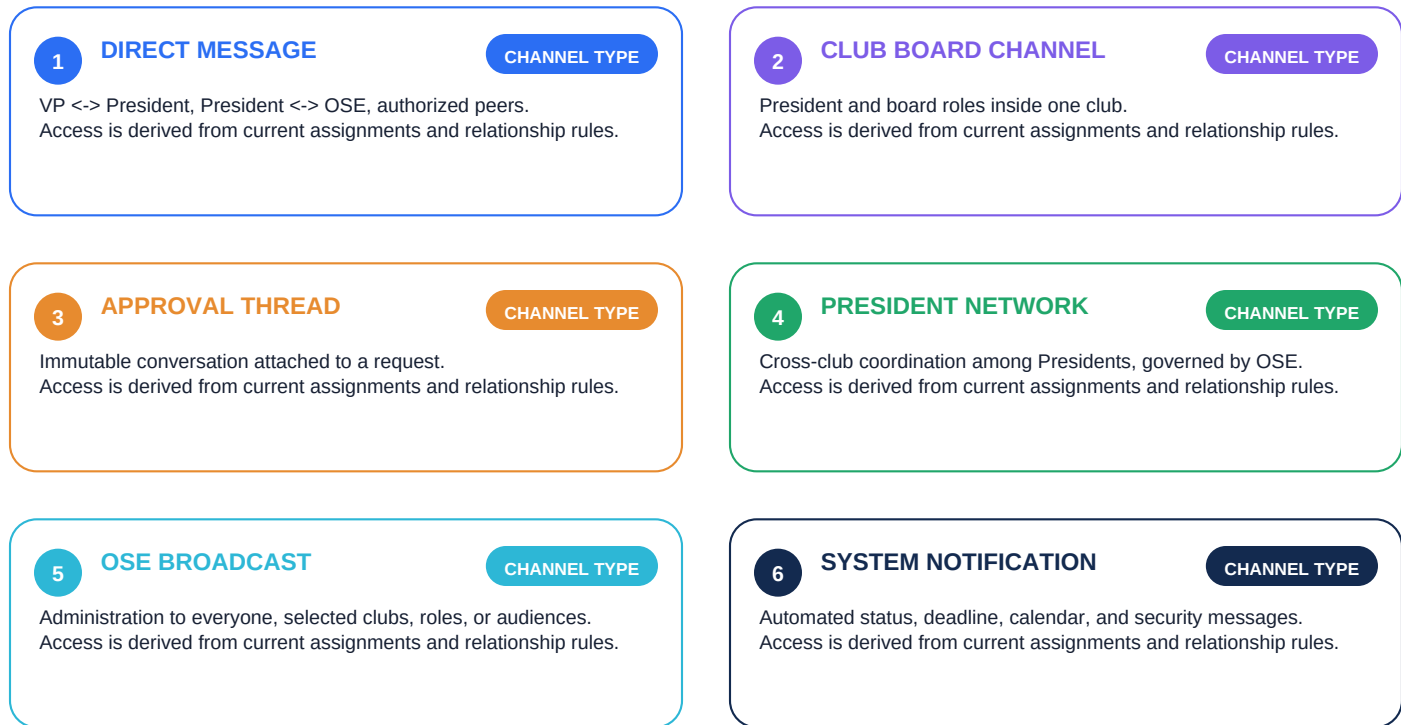
Conflict type	Example	System behavior
Hard: same reservable resource	Two approved events in the same room and time	Block submission or route as exception; never silently double-book
Hard: institutional blackout	Event during prohibited time or required academic window	Block or require OSE override
Soft: audience collision	Two major MBA-wide events at the same time	Warn, show estimated overlap, allow governed decision
Soft: similar category	Multiple recruiting or career events clustered together	Recommend alternatives without automatic rejection
Informational	Different audiences or virtual events overlap	Display context; no action required

Event entity and integration boundary

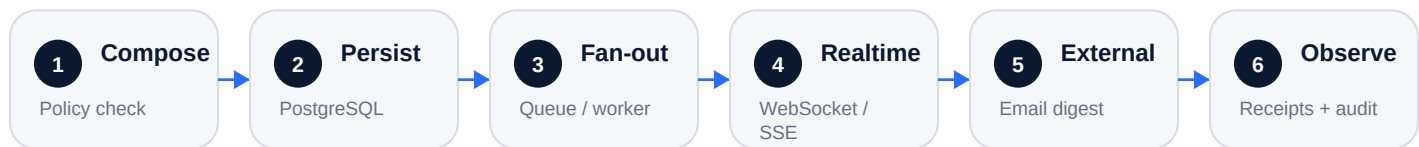
Event: event_id, tenant_id, organization_id, owner_role_seat_id, title, description, start_at, end_at, timezone, venue/resource, audience, capacity, status, approval_request_id, recurrence_rule, visibility, conflict_summary, created_by, updated_at.

Integrations: v1 can export ICS and send email notifications. Outlook / Google Calendar two-way sync is an adapter after the internal event and approval model is stable.

Messaging, InMail, and notification architecture



Delivery and storage pipeline



Messaging entities

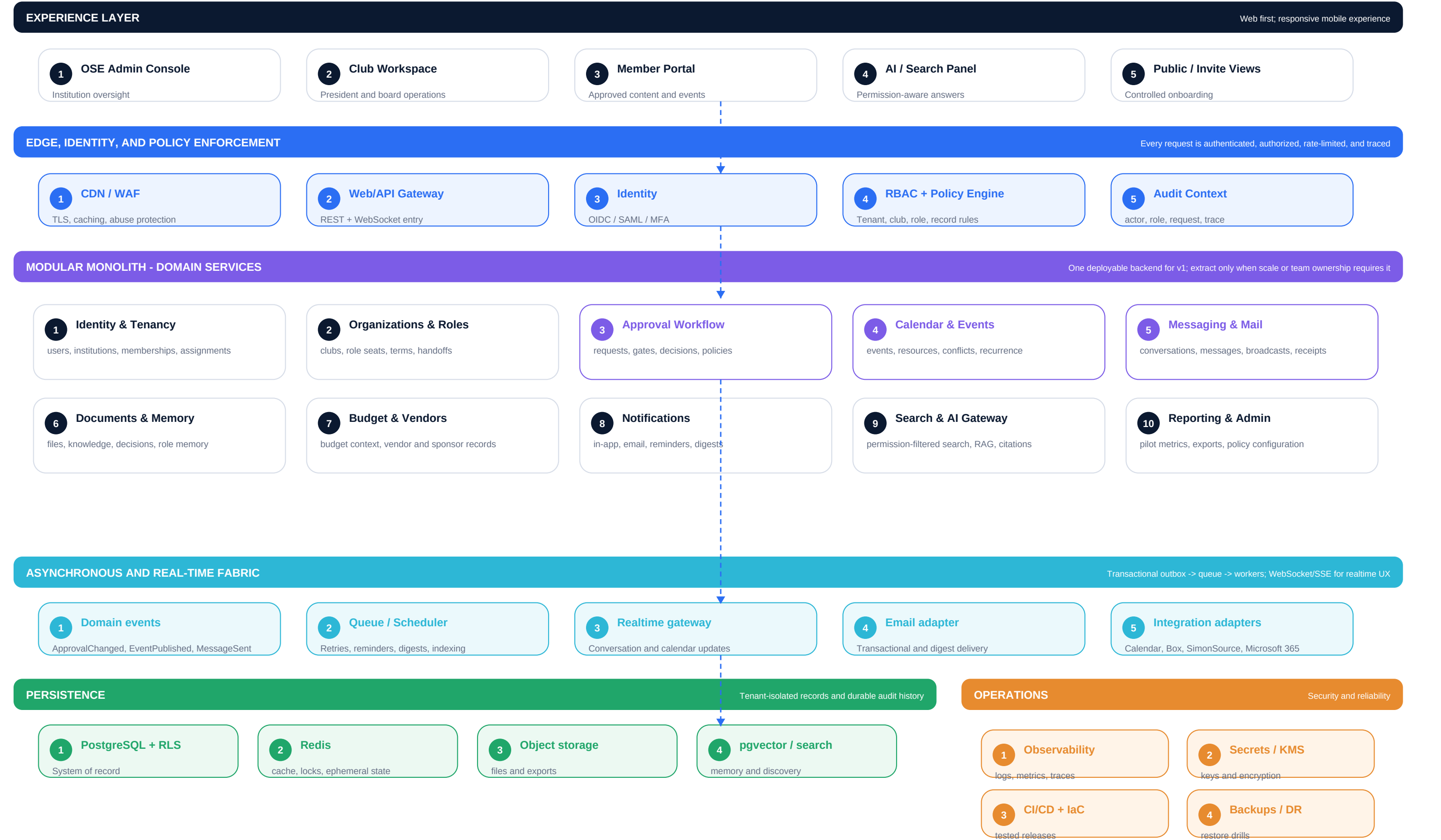
Entity	Purpose	Critical fields
conversation	Container for DM, channel, approval, broadcast, or record thread	type, tenant_id, organization_id, linked_record_type/id, visibility
participant	User / role membership in a conversation	user_id, role_context, joined_at, last_read_message_id
message	Immutable communication entry	sender_id, body, format, created_at, edited_at, reply_to_id
attachment	Files attached with inherited authorization	object_key, mime_type, checksum, sensitivity
delivery	Per-recipient delivery and read state	channel, delivered_at, read_at, failure_reason

Non-negotiable communication rules

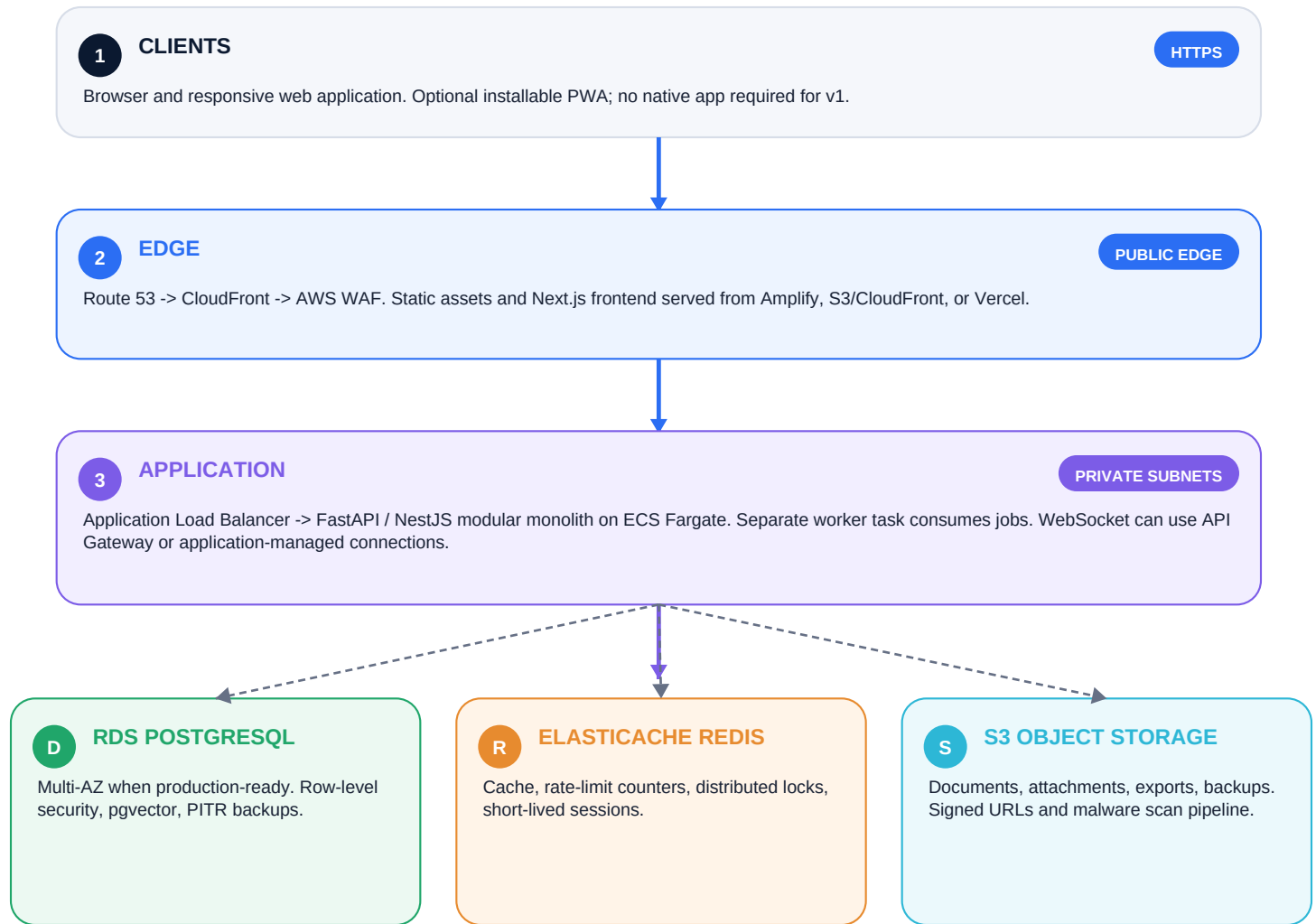
- Authorization is evaluated at send time and read time; losing a role may remove access to future private content.
- OSE institutional access is explicit and audited. The UI must not imply private end-to-end messaging when administrators can access records.
- Approval messages cannot be deleted; corrections are appended and linked.

• Broadcasts require audience preview, confirmation, delivery summary, and throttling.

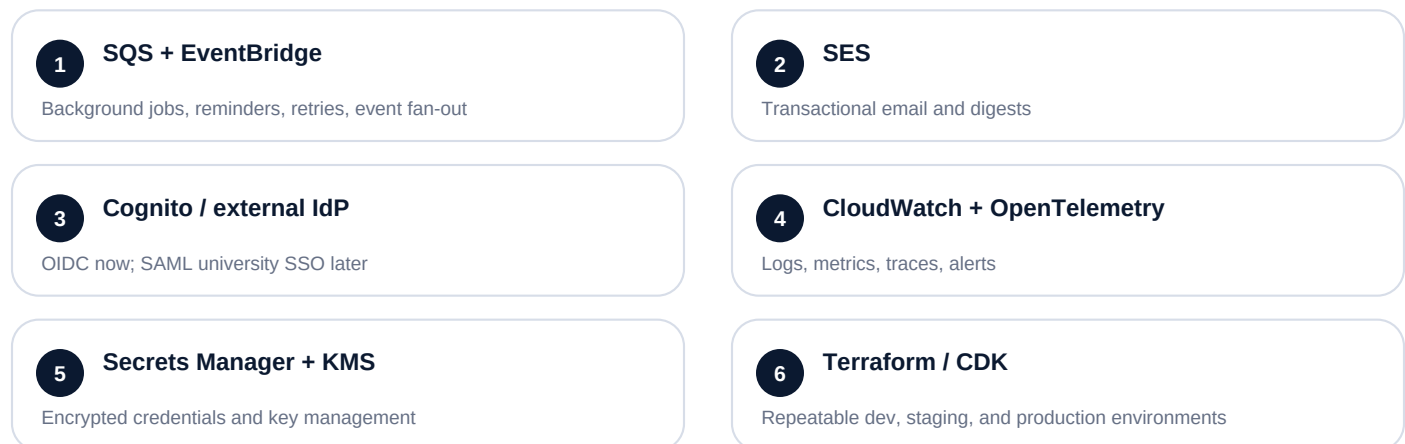
Tenure master system architecture - one view



Recommended AWS deployment topology



Async processing and external services



Architectural recommendation: do not deploy Kubernetes for the Simon pilot. ECS Fargate or an equivalent managed container platform reduces operational burden while preserving portability.

Domain modules and ownership boundaries

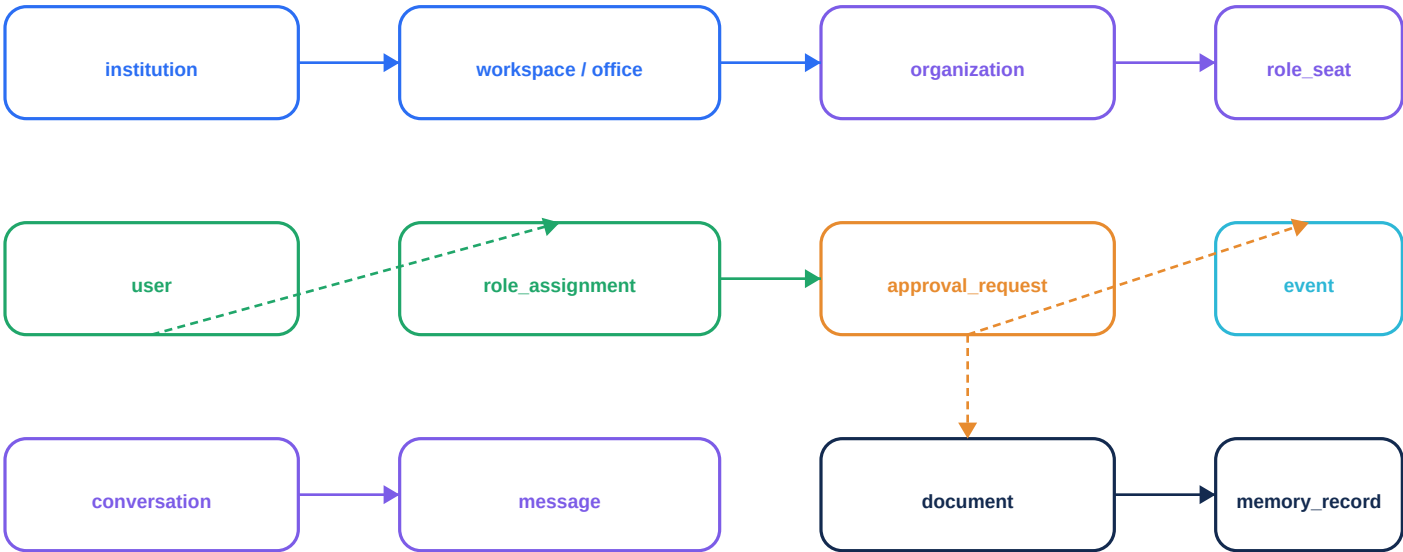
Module	Owns	Publishes	Depends on
Identity & Tenancy	institutions, users, memberships, sessions	UserActivated, AssignmentChanged	Identity provider
Organizations & Roles	clubs, role seats, role assignments, handoffs	RoleAssigned, HandoffCompleted	Identity & Tenancy
Approvals	requests, steps, decisions, policy snapshots	ApprovalSubmitted, ApprovalChanged	Organizations, Calendar, Notifications
Calendar & Events	events, resources, conflicts, recurrences	EventProposed, EventPublished, ConflictDetected	Approvals, Organizations
Messaging	conversations, participants, messages, receipts	MessageSent, BroadcastPublished	Identity, Organizations
Documents & Memory	files, documents, decisions, role memory	DocumentIndexed, MemoryAdded	Object storage, Search / AI
Budget & Vendors	budgets, transactions context, vendors, sponsors	BudgetUpdated, VendorAdded	Organizations, Approvals
Notifications	preferences, delivery jobs, templates	NotificationDelivered, DeliveryFailed	All domain events
Search & AI	indexes, embeddings, queries, citations	SearchIndexUpdated, AIQueryLogged	Documents, Organizations, policy engine
Reporting & Admin	metrics, exports, configuration, audit views	ReportGenerated, PolicyChanged	All modules, audit log

Consistency pattern

Strong transaction: business record + state transition + outbox event commit in one PostgreSQL transaction. **Eventual processing:** notifications, search indexing, AI embeddings, email, analytics, and integrations consume the outbox asynchronously with idempotency keys and retries.

Why: a user must never see an approval as completed while the database still considers it pending. Secondary systems can catch up safely without blocking the interactive request.

Canonical data model



Core tables and invariants

Table	Critical invariant
institutions / workspaces / offices	Every domain row resolves to a tenant boundary; cross-tenant queries are denied by default.
organizations	Club identity persists across academic years; status is active, archived, or pending.
role_seats	Durable position inside an organization; role type, scope, and template are versioned.
role_assignments	Temporal person-to-seat link with start/end dates; overlapping active occupants require explicit policy.
approval_requests / approval_steps	State transitions are append-only and store actor, role context, reason, and policy snapshot.
events / resources / conflicts	Timestamps use UTC plus named timezone; recurrence and exception records are explicit.
conversations / participants / messages	Tenant and visibility inherited from conversation; messages are immutable with correction metadata.
documents / memory_records / citations	Source provenance, version, sensitivity, and authorization metadata required for retrieval.
audit_events	Append-only, tamper-evident, searchable by actor, resource, action, and trace.

Tenant isolation key

Required on all business records: `tenant_id`. Club-owned records also require `organization_id`. Policies may grant institution-wide access to OSE, but the database query must still express the authorized tenant and scope explicitly.

Security, privacy, and audit architecture

1 IDENTITY

CONTROL DOMAIN

OIDC / SAML, verified email domains, MFA for OSE, session rotation, device/session revocation.

2 AUTHORIZATION

CONTROL DOMAIN

RBAC baseline plus policy conditions, row-level security, object sensitivity, workflow-state checks.

3 DATA PROTECTION

CONTROL DOMAIN

TLS, encryption at rest, KMS-managed keys, signed file URLs, restricted backups, secret rotation.

4 APPLICATION SECURITY

CONTROL DOMAIN

Input validation, CSRF protections, CSP, rate limiting, dependency scanning, secure headers.

5 AUDITABILITY

CONTROL DOMAIN

Sensitive reads, exports, approvals, role changes, impersonation, policy changes, and OSE overrides logged.

6 PRIVACY OPERATIONS

CONTROL DOMAIN

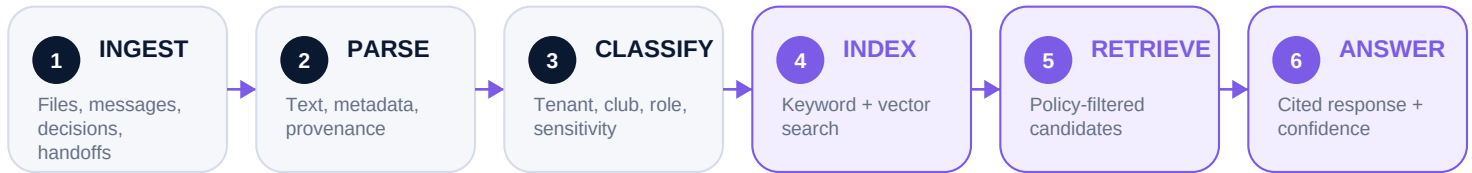
Data classification, retention schedules, deletion workflows, access reviews, incident response.

Priority threat scenarios and mitigations

Threat	Mitigation
VP attempts to access another club budget	Organization-scoped policy, tenant filter, row-level security, denied-access audit.
Former officer retains access after turnover	Assignment expiry, session invalidation, event-driven permission cache purge.
Message or attachment link is forwarded	Short-lived signed URL, recipient authorization on download, sensitivity-aware preview.
President self-approves a request	Separation-of-duty rule evaluated in workflow engine and database constraint where possible.
OSE override is abused	Step-up authentication, mandatory reason, immutable audit, periodic override review.
AI answer reveals restricted context	Authorization-filtered retrieval before ranking; citations checked against caller permissions.

Important: Tenure should design for FERPA-adjacent institutional expectations, but should not claim FERPA, SOC 2, or other formal compliance solely from architecture. Legal review, policies, operational evidence, and independent controls are required.

Institutional memory and AI/RAG architecture



Memory scopes

1 Institution / OSE

Official policies, training, deadlines, approved templates, institutional decisions.

2 Club

Club history, recurring events, sponsor relationships, past decisions, constitutions.

3 Role seat

President, VP Finance, VP Marketing, outreach, and other durable role knowledge.

4 Record-linked

Context attached to an event, approval, vendor, budget item, or conversation.

Answer path and guardrails

1. Resolve caller

Tenant, club assignments, role seats, current term, and sensitivity grants.

2. Filter before retrieval

Apply SQL / index filters before semantic ranking; never retrieve then redact as the primary control.

3. Require provenance

Every answer sentence that states institutional or historical fact links to a source record.

4. Expose uncertainty

Low-confidence or conflicting records return a qualified answer and direct the user to OSE.

5. Log responsibly

Store query, sources, model/version, outcome, and feedback without retaining unnecessary sensitive text.

AI non-goals for v1

No autonomous policy approval. No fabricated institutional rules. No unrestricted cross-club retrieval. No password storage in the knowledge base. No model training on tenant data without an explicit, reviewed agreement.

Reliability, observability, and delivery engineering

1 AVAILABILITY

Target 99.5% during pilot; scheduled maintenance communicated.

2 LATENCY

P95 interactive API < 500 ms excluding AI generation.

3 DURABILITY

No acknowledged approval or message lost; database PITR enabled.

4 RECOVERY

Documented restore procedure; pilot RPO <= 24 h, RTO <= 4 h.

Observability model

Signal	Examples	Alert / owner
Metrics	request latency, error rate, queue age, WebSocket connections, approval throughput, email failure	Threshold and anomaly alerts to engineering
Logs	structured application, policy deny, security, worker, integration logs with trace_id	Searchable; sensitive fields redacted
Traces	browser/API -> domain module -> database/queue/external service	Used for slow transactions and failed workflows
Product telemetry	active clubs, handoff completion, questions answered, calendar conflicts prevented	Reviewed with product and OSE; privacy-minimized
Audit events	role changes, approval actions, exports, OSE overrides, sensitive reads	Administrative review and incident investigation

CI/CD and environment strategy

Pull request: lint -> unit tests -> type checks -> migration validation -> dependency and secret scan -> ephemeral preview. **Merge:** build immutable container -> deploy staging -> integration tests -> manual production approval -> gradual rollout -> smoke tests -> rollback if health

1 LOCAL

Docker Compose; seeded test data; no production secrets.

2 DEV

Shared integration environment; synthetic university tenant.

3 STAGING

Production-like configuration and migration rehearsal.

4 PRODUCTION

Separate account/project, restricted access, backups, alerts.

Resilience patterns

- Idempotency keys for request creation, approval transitions, message sends, calendar publishing, and external webhooks.
- Transactional outbox and retry queues with exponential backoff, dead-letter handling, and replay tooling.
- Feature flags for new workflows and integrations; schema migrations remain backward-compatible during deployment.
- Backups are not considered valid until restore tests are performed and documented.

API and event contract examples

Representative REST endpoints

Method	Endpoint	Purpose / authorization
GET	/v1/organizations	OSE: all; users: only organizations they can access
POST	/v1/organizations/{id}/role-assignments	President or OSE; policy validates role and term
POST	/v1/approval-requests	Authorized VP / President; type-specific validation
POST	/v1/approval-requests/{id}/transitions	Expected-state check, separation of duty, idempotency key
GET	/v1/calendar?from=&to;=&scope;=	Permission-aware institutional or club event feed
POST	/v1/events/{id}/conflict-check	Returns hard, soft, and informational conflicts
POST	/v1/conversations/{id}/messages	Participant and record-scope authorization
POST	/v1/search/query	Policy-filtered keyword / vector / AI retrieval
GET	/v1/audit-events	OSE / security only; filtering and export audited

Domain event envelope

```
{
  "event_id": "evt_...",
  "event_type": "ApprovalStateChanged.v1",
  "occurred_at": "2026-09-14T18:42:11Z",
  "tenant_id": "inst_simon",
  "organization_id": "club_finance",
  "actor": {"user_id": "usr_...", "role_seat_id": "president"},
  "subject": {"type": "approval_request", "id": "apr_..."},
  "data": {"from": "PENDING_PRESIDENT", "to": "PENDING_OSE"},
  "trace_id": "trc_...", "schema_version": 1
}
```

Contract principles

- Version APIs and events deliberately; consumers ignore unknown additive fields.
- Use idempotency keys on mutating endpoints and optimistic concurrency on workflow transitions.
- Return machine-readable policy denial codes, not only generic 403 responses.
- Do not expose database identifiers or storage paths that bypass authorization.

MVP build sequence and pilot acceptance

1

WEEK 1 - Foundation

DELIVERABLE

Architecture repo, environments, identity, tenant model, organization and role-seat schema.

2

WEEK 2 - RBAC + roster

DELIVERABLE

Assignments, permissions, OSE admin, President and VP access, audit context.

3

WEEK 3 - Approvals

DELIVERABLE

Request types, President gate, OSE gate, state history, notifications.

4

WEEK 4 - Calendar

DELIVERABLE

Event model, shared views, conflict checks, approval-linked publishing.

5

WEEK 5 - Messaging

DELIVERABLE

DMs, board channels, approval threads, OSE broadcasts, receipts.

6

WEEK 6 - Documents + memory

DELIVERABLE

File storage, role memory, handoff records, search metadata.

7

WEEK 7 - AI + reporting

DELIVERABLE

Permission-aware retrieval, citations, pilot dashboards, exports.

8

WEEK 8 - Hardening

DELIVERABLE

Security review, load tests, migration rehearsal, onboarding, support runbook.

Pilot acceptance criteria

Area	Minimum acceptance
Organization setup	30+ clubs loadable; 4-6 pilot clubs active; 3-5 board roles per club supported without custom code.
RBAC	OSE sees all; President controls own club; VPs limited by function; cross-club isolation tests pass.
Approvals	VP -> President -> OSE flow works with returns, rejection, notifications, audit, and no self-approval.
Calendar	All clubs and OSE can view authorized events; hard conflicts are blocked or governed; soft conflicts are visible.
Messaging	Required relationship paths work; broadcasts and approval threads are auditable; read states function.
Reliability	Backups, restore test, monitoring, error handling, support owner, and incident process documented.

Pilot boundaries, product metrics, and next decisions

BUILD FOR FALL 2026

- Institution, school/office, club, role-seat hierarchy
- OSE full-access administration and audit
- President and functional VP permissions
- Approval requests and state history
- Unified calendar with conflict checks
- In-system messaging and broadcasts
- Documents, role memory, and handoffs
- Basic search / source-cited AI
- Pilot reporting and export

DEFER UNTIL VALIDATED

- Full financial ledger or payment processing
- University room reservation replacement
- Complex procurement and reimbursement engine
- Native iOS / Android applications
- Kubernetes and independent microservices
- Deep SimonSource / Banner / Canvas integrations
- Autonomous AI approvals or policy decisions
- Cross-university marketplace features
- Advanced predictive analytics

Metrics that prove product value

1 Adoption

Active clubs, active officers, weekly active users, completion of onboarding.

2 Workflow efficiency

Approval turnaround, returned requests, reminder volume, repeated questions.

3 Calendar value

Conflicts detected, conflicts resolved before approval, cross-club coordination.

4 Continuity

Role-memory records, completed handoffs, source retrieval success, missing context.

5 Communication

Message delivery/read rates, OSE broadcast reach, email duplication reduced.

6 Trust

Authorization failures, audit completeness, incident count, satisfaction and recommendation.

Decisions to confirm with Simon OSE before implementation

- Which approval types require President only, OSE final approval, or no approval?
- Which event conflicts are hard blocks versus warnings, and is venue inventory in scope?
- Can OSE administrators access all internal club messages, and how must this be disclosed?
- Which VP role templates should ship by default, and may clubs create custom roles?
- What retention, export, and deletion expectations apply to student and organizational records?
- Which identity method is acceptable for the pilot before university SSO integration?

Glossary and source context

Term	Definition
Tenant	The institution or contracted school boundary that owns users, clubs, policy, and data.
Organization / Club	A student-led club or association managed under OSE oversight.
Role seat	A durable position such as President or VP Finance that remains when the occupant changes.
Role assignment	Time-bounded link between a user and a role seat.
Functional scope	Subset of club capabilities granted to a VP / Chair based on their responsibility.
Approval request	Structured workflow record submitted through required governance gates.
Institutional calendar	Unified view of proposed and approved events across clubs and OSE.
Conversation	Permission-controlled container for messages, approval threads, channels, or broadcasts.
Institutional memory	Verified historical context attached to institution, club, role, or business record.
Policy engine	Authorization component evaluating role, tenant, club, sensitivity, relationship, and workflow state.
Transactional outbox	Pattern that commits a domain event with the business transaction before asynchronous delivery.
RAG	Retrieval-augmented generation: AI answers based on permission-filtered source records and citations.

Source context

This blueprint synthesizes the Tenure product direction, the founders' stated organizational requirements, and the Simon Business School Office of Student Engagement discovery questionnaire. The questionnaire identifies onboarding, repeated explanations, fragmented records, leadership handoff, budget/vendor/event memory, communication overload, and a Fall 2026 pilot as primary product context.

Architecture status

Recommended baseline: this document is a design blueprint for implementation planning. Cloud provider, identity vendor, final approval policy, messaging visibility, retention rules, and integration commitments must be confirmed before production launch.